



# CAMPAIGNLIFT

## Project Report

AI DEV FEST 2026 AI Hackathon • Track 04 — Growth & Campaign Intelligence

|        |                                           |
|--------|-------------------------------------------|
| TEAM   | Deception                                 |
| TRACK  | Track 04 — Growth & Campaign Intelligence |
| STATUS | Submission-ready technical report         |
| DATE   | October 2026                              |

*A concise technical account of the problem, uplift methodology, architecture, evaluation, deployment, and operational limits of the CampaignLift prototype.*

## 1. Executive Summary

CampaignLift is a decision-support system for campaign managers in Mobile Financial Services (MFS). The project addresses a practical problem: a customer who is already likely to transact may not need an incentive, while another customer may transact only because of the offer.

The system therefore uses uplift modelling to estimate the difference between a customer's expected outcome with an offer and without an offer. The resulting scores are used for audience prioritisation, strategy comparison, and budget allocation.

The current implementation is a synthetic-data prototype. It demonstrates the complete decision workflow and provides measured model results, but it is not presented as a production MFS deployment.

## 2. Problem Statement

MFS campaigns commonly target customers with high predicted response. This can lead to spending on customers who would have completed the transaction without an incentive, while customers who are genuinely persuadable may be overlooked.

The project focuses on two related problems:

## 3. Decision Approach

A response model answers the question: Who is likely to transact after receiving an offer? CampaignLift instead asks: Who is more likely to transact because of the offer?

The project represents this difference as an uplift score:

$$\tau(X) = P(\text{Transact} \mid X, \text{Offer}) - P(\text{Transact} \mid X, \text{No Offer})$$

This separates three useful groups: Persuadables, who have positive incremental response; Sure Things, who are likely to transact without the offer; and Sleeping Dogs, whose estimated response may decrease under treatment.

A simple example illustrates the distinction. If a customer has a 91% probability of transacting without an offer and 93% with an offer, the incremental lift is only +2 percentage points. A customer moving from 18% to 72% has +54 points and is therefore much more valuable to target.

- Budget efficiency: distinguish naturally active customers from customers whose behaviour is likely to change because of an offer.
- Customer treatment: avoid unnecessary incentives and identify segments where an offer may have little effect or a negative effect.

## 4. System Architecture

The implementation uses a browser-based React/TypeScript interface, an Nginx web and reverse-proxy layer, and an internal FastAPI service. Model artifacts and feature data are packaged with the backend, while campaign state is persisted through SQLite.

| CAMPAIGN MANAGER / BROWSER                         |
|----------------------------------------------------|
| React 19 + TypeScript • Analytics Workstation      |
| Nginx • HTTPS / Static Assets / API Reverse Proxy  |
| FastAPI • Decision Support API                     |
| Uplift Model + Feature Table                       |
| SQLite • Campaign State                            |
| Gemini Copilot • Optional, server-side integration |

The backend is not exposed directly to the public host. Nginx provides the public HTTP/HTTPS entry point and forwards API requests to the backend on the internal Docker network. This keeps the application boundary simple while allowing the frontend and backend to be deployed as independent images.

## 5. Data and Modelling

The simulator represents an MFS-style customer base with 25,000 customers and more than 500,000 transactions. The generated data includes customer tenure, transaction history, merchant category behaviour, channel affinity, campaign exposure, and treatment-related variables.

Feature construction is performed before treatment assignment. The project uses a timestamp rule (timestamp < assigned\_at) and schema assertions to prevent hidden causal parameters from entering the training features.

Model selection considered Logistic Baseline, Logistic T-Learner, LightGBM T-Learner, and LightGBM S-Learner candidates. The final reported model is the LightGBM S-Learner (U2).

## 6. Model Evaluation

Evaluation used a 60/20/20 train/validation/test split. The test split contained N = 2,530 records. The table below reports the measured values supplied by the project evaluation artifacts.

| Metric                         | Validation                                  | Test                                        |
|--------------------------------|---------------------------------------------|---------------------------------------------|
| Winning architecture           | LightGBM S-Learner (U2)                     | LightGBM S-Learner (U2)                     |
| Model version                  | cl-model-ml_dev_20261006-lgbm_s_learner-r01 | cl-model-ml_dev_20261006-lgbm_s_learner-r01 |
| Qini score                     | 48.4578                                     | 65.2257                                     |
| Normalized Qini (AUUC)         | 0.1342                                      | 0.1755                                      |
| Top 10% incremental rate       | 0.1863 (+18.6% lift)                        | 0.1873 (+18.7% lift)                        |
| Average treatment effect (ATE) | -0.0198                                     | -0.0076                                     |

| Metric             | Validation | Test |
|--------------------|------------|------|
| Inference fit time | 0.057 s    | —    |

The LightGBM S-Learner produced the strongest reported AUUC result among the evaluated candidates. The values above are measured project results; they should not be interpreted as evidence of production performance on real MFS populations.

## 7. Application Workflow

1. Campaign setup — define the campaign objective, offer, channel, incentive cost, and budget.
2. Scoring — evaluate eligible customers and return treatment probability, control probability, uplift, and ranking information.
3. Strategy comparison — compare Random, Response, and Uplift strategies under the same campaign constraints.
4. Budget allocation — select customers subject to the campaign budget and negative-uplift exclusion settings.
5. Customer explanation — expose feature contributions and reason codes for individual recommendations.
6. Experiment view — present randomized treatment and control results with support indicators for slices.
7. Copilot — use Gemini to summarize a campaign run from verified backend context when configured.

## 8. Responsible Use and Limitations

The system is designed as decision support. It does not send customer messages or make financial disbursements automatically.

The current dataset is synthetic and cannot establish how the model would perform on real customer populations. The simulated experiment is also not a substitute for a controlled commercial experiment.

Feature contributions are explanations of model behaviour and should not be interpreted as proof of causal influence for an individual feature.

Before operational use, the model would require validation on governed data, a real randomized holdout, appropriate audit logging, and review against applicable financial and data-protection requirements.

## 9. Deployment and Operational Setup

The application is packaged as separate frontend and backend Docker images. The frontend image serves the React build through Nginx and terminates HTTPS at the edge. Requests under `/api/` are reverse-proxied to the backend over the internal Docker network.

The backend image contains the FastAPI application, ML dependencies, model artifacts, and feature fixtures. The backend port is internal to the Compose network rather than published directly to the host.

For the current deployment pattern, images are published to GitHub Container Registry and started through Docker Compose on a VPS/AWS environment. Runtime secrets such as the Gemini API key are supplied at runtime and are not embedded in the images.

### Deployment components

| Layer        | Implementation       | Role                                          |
|--------------|----------------------|-----------------------------------------------|
| Edge         | Nginx / HTTPS        | Static delivery, TLS, reverse proxy           |
| Application  | FastAPI / Python     | Decisioning and inference API                 |
| Runtime      | Docker Compose / VPS | Container orchestration and service isolation |
| State        | SQLite volume        | Campaign state persistence                    |
| AI assistant | Gemini API           | Optional grounded campaign summarisation      |

## 10. Requirements for Real-World Deployment

- Governed data access — replace synthetic feature tables with secured, PII-tokenized MFS data.
- Randomized holdout — run a genuine treatment/control experiment, with a predefined control allocation, to measure incremental outcomes.
- Model monitoring — track calibration, uplift quality, segment stability, drift, and campaign-level outcomes over time.
- Auditability — retain campaign configuration, model version, dataset version, decision inputs, and resulting actions.
- Compliance review — align data handling and campaign decisioning with applicable Bangladesh Bank and consumer data-protection requirements.

## 11. Project Resources

Source, deployment, and published container resources for the current prototype.

### REPOSITORY

<https://github.com/mosabbir-maruf/CampaignLift-Hackathon>

### LIVE APPLICATION

<https://devtree.online/>

### FRONTEND IMAGE

[ghcr.io/mosabbir-maruf/campaignlift-frontend:latest](https://ghcr.io/mosabbir-maruf/campaignlift-frontend:latest)

### FRONTEND PACKAGE

<https://github.com/users/mosabbir-maruf/packages/container/package/campaignlift-frontend>

### BACKEND IMAGE

[ghcr.io/mosabbir-maruf/campaignlift-backend:latest](https://ghcr.io/mosabbir-maruf/campaignlift-backend:latest)

### BACKEND PACKAGE

<https://github.com/users/mosabbir-maruf/packages/container/package/campaignlift-backend>

### CONCLUSION

CampaignLift demonstrates an end-to-end uplift-based campaign decision workflow on synthetic MFS-style data. The prototype shows how campaign selection can be framed around incremental impact rather than response propensity, while making the model, data, deployment, and operational limits explicit.